

Lab 3: Architecture Selection Justification

Problem Statement #55: Book Club Reading Challenge & Discussion Portal | PES1UG24CS424

Architecture Selection: We chose **Client-Server Architecture** for the **Book Club Reading Challenge & Discussion Portal** System.

Architectural Choice

The portal has two tiers. The **Client Tier** is the Web Client Component running in the member's browser. The **Server Tier** is a central portal server made of an API Server Component (the single entry point), four service components (Auth, Discussion, Progress and Poll) and one Database Component. The client reaches the server only through IPortalAPI, the API Server calls each service through its own interface, and the Discussion Service asks the Progress Service through IReadStatus whether a chapter is read before it unmask spoilers.

Two Reasons

- 1. The key requirements need one authoritative source of truth.** FR-005 and NFR-001 demand exactly one vote per member and a live tally, and FR-001 and FR-004 make spoiler visibility depend on each member's chapter-read state. A central server keeps votes, read status and comments in one database, so consistency is straightforward. Microservices would have to keep separate data stores in sync, which is exactly where duplicate votes and stale spoiler state creep in.
- 2. It matches how a book club is used.** Members are readers on browsers or phones who log in at different times and read at different paces, and a club is small. A thin browser client needs no installation and every fix is deployed once on the server. The operational cost of microservices is not justified at this size, and the service components still stay separate behind interfaces, so any one of them can be replaced later.

Security Advantage

All enforcement happens on the server, so the client cannot bypass it. The API Server verifies the session once (ISessionService) before any state-changing request such as posting a comment or voting, matching the "Verify User Session" step in the use-case model. Spoiler-tagged text is masked on the server before the response is sent, so hidden content never reaches a member who has not revealed it or marked the chapter read (FR-001). Only the Discussion Lead role can create polls, and a unique (poll, member) constraint in the database rejects duplicate votes even if two requests arrive together (NFR-001).

Performance Benefit

The server holds the single authoritative tally. Each vote is one write, after which the server pushes the new tally over WebSocket to connected clients, giving the real-time updates NFR-001 asks for without any cross-service network hops. Thin clients also do little work, so the portal stays responsive on low-powered phones.

Trade-off Accepted

Client-server has a single point of failure and a scaling bottleneck. For a club-sized portal this is acceptable, and NFR-002 is addressed by keeping every committed comment and progress entry in the durable database so it survives a restart, together with regular backups and automatic restart of the server, which is enough to reach the 99.5% monthly uptime target.